



CONCORRÊNCIA E MULTITHREADING

AULA 06 - COMPLETABLEFUTURE



AGENDA

- ❑ CompletableFuture: Programação Assíncrona e Reativa
- ❑ Superando as limitações do Future.
- ❑ Compondo, combinando e tratando erros em operações assíncronas.



CRIANDO UM COMPLETABLEFUTURE

- ❑ `runAsync(Runnable)`: Executa uma tarefa assíncrona que não retorna valor. Retorna `CompletableFuture<Void>`.
- ❑ `supplyAsync(Supplier<T>)`: Executa uma tarefa assíncrona que retorna um valor. Retorna `CompletableFuture<T>`.
- ❑ **Executor Padrão**: Por padrão, usa o `ForkJoinPool.commonPool()`. Uma sobrecarga permite fornecer um Executor customizado.



ENCADEAMENTO E TRANSFORMAÇÃO DE RESULTADOS

- ❑ `thenApply(Function<T,U>)`: Transforma o resultado T em um resultado U de forma síncrona. Análogo ao `map()` do Stream.
- ❑ `thenRun(Runnable)`: Executa uma ação após a conclusão, sem usar o resultado.
- ❑ `thenAccept(Consumer<T>)`: Consome o resultado T sem retornar nada. Uma ação terminal.
- ❑ Variantes `...Async`: `thenApplyAsync`, etc., executam o estágio em um pool de threads (geralmente o `commonPool`).



THENAPPLY VS. THENCOMPOSE

- ❑ `thenApply`: Para mapeamento simples. $T \rightarrow U$.
 - ❑ `getUserById(id).thenApply(User::getEmail)`.
 - ❑ O método `getEmail` retorna uma `String`, não um `CompletableFuture`.
- ❑ `thenCompose`: Para encadear operações que já são assíncronas. $T \rightarrow \text{CompletableFuture}\langle U \rangle$. Ele "achata" o resultado para `CompletableFuture\langle U \rangle`. Análogo ao `flatMap()` do `Stream`.



COMBINANDO MÚLTIPLOS FUTUROS

- ❑ `thenCombine(other, BiFunction)`: Combina os resultados de dois futuros independentes quando ambos estiverem concluídos.
- ❑ `allOf(futures...)`: Cria um `CompletableFuture<Void>` que é concluído quando todos os futuros fornecidos são concluídos.
 - ❑ `Stream.of(f1, f2, f3).map(CompletableFuture::join).collect(Collectors.toList())`
- ❑ `anyOf(futures...)`: Cria um `CompletableFuture<Object>` que é concluído quando qualquer um dos futuros fornecidos é concluído.

TRATAMENTO DE ERROS ROBUSTO

- ❑ `exceptionally(Function<Throwable, T>)`: Fornece um valor de fallback caso o pipeline falhe. Análogo a um bloco catch.
- ❑ `handle(BiFunction<T, Throwable, U>)`: Processa tanto o resultado de sucesso (T) quanto a exceção (Throwable). Análogo a um bloco finally que pode retornar um valor.



ATÉ A PRÓXIMA!